

URL Shortener - Hash + Collision Resolution

1. Overview

To generate a short URL, we need a **hash function** that converts a long URL into a **fixed-length short string (hashValue)**.

Requirement:

- Short URL length $\approx$ **7 characters**
- Character set: **[0-9, a-z, A-Z] (Base62)**

2. Basic Approach

Use well-known hash functions:

- **CRC32**
- **MD5**
- **SHA-1**

Example input:

None

https://en.wikipedia.org/wiki/Systems_design

The following table compares the hash results after applying different hash functions on this URL: https://en.wikipedia.org/wiki/Systems_design.

Hash function	Hash value (Hexadecimal)
CRC32	5cb54054
MD5	5a62509a84df9ee03fe1230b9df8b84e
SHA-1	0eeae7916c06853901d9ccbefbfcaf4de57ed85b

Problem:

- Even the **smallest hash (CRC32)** produces output **longer than 7 characters**
- Hence, direct usage is **not suitable**

3. Solution: Truncation

Step 1: Generate hash

None

```
hashValue = hash(longURL)
```

Step 2: Take first 7 characters

None

```
shortURL = hashValue[0:7]
```

4. Problem: Hash Collision

Collision occurs when:

None

Different long URLs → Same shortURL

Example:

None

URL1 → abc1234

URL2 → abc1234 ❌ collision

5. Collision Resolution Strategy

Solution: **Recursive appending**

Process:

1. Generate initial short URL:

None

shortURL = first 7 chars of hash

- 2.

Check in database:

None

if shortURL exists:

 modify input (append predefined string)

 regenerate hash

 repeat

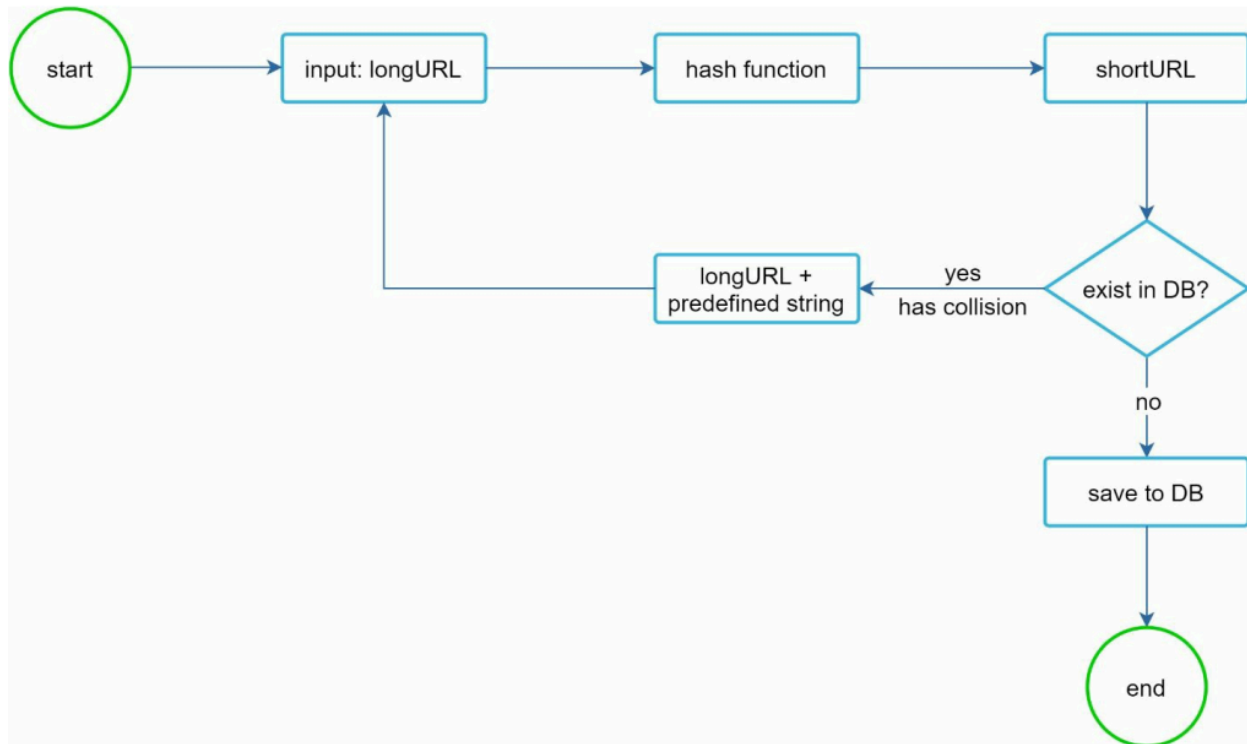
- 3.

Continue until:

None

unique shortURL is found

Example Flow:



None

URL → hash → abc1234 ❌ exists

URL + "1" → hash → xyz5678 ❌ exists

URL + "2" → hash → pqr9123 ✅ unique

6. Drawback

Expensive operation:

- Requires **database lookup for every attempt**
- Multiple retries possible in case of collisions

7. Optimization: Bloom Filter

To reduce database queries:

- Use **Bloom Filter** to check existence before DB lookup

Benefits:

- Fast (in-memory)
- Space efficient
- Reduces unnecessary DB hits

Trade-off:

- May give **false positives**, but never false negatives

8. Key Takeaways

- Hash functions alone produce **long outputs**
- Truncation introduces **collision risk**
- Collision handled via:
 - Recursive hashing
 - Existence checking
- Bloom filters improve performance